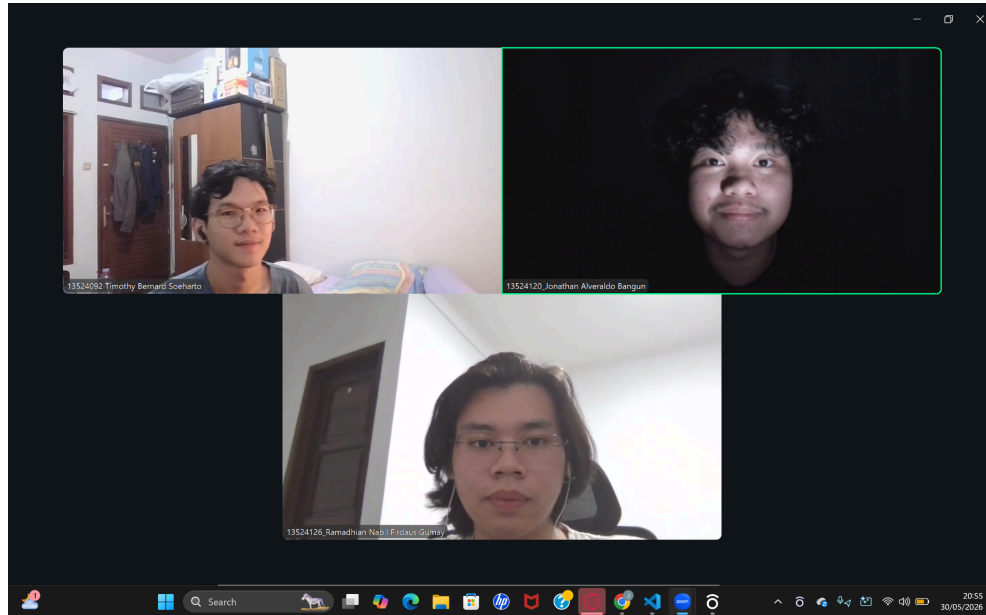


TUGAS BESAR III

IF2211 STRATEGI ALGORITMA

Implementasi Pattern Matching pada Chromium Browser Extension untuk Deteksi Judi Online



Timothy Bernard Soeharto - 13524092

Jonathan Alveraldo Bangun - 13524120

Ramadhian Nabil Firdaus Gumay - 13524126

Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

2026

PERNYATAAN

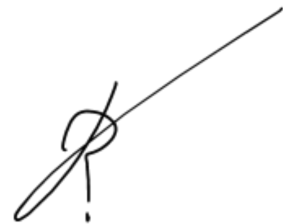
Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.



Timothy Bernard Soeharto - 13524092



Jonathan Alveraldo Bangun - 13524120



Ramadhian Nabil Firdaus Gumay - 13524126

DAFTAR ISI

PERNYATAAN.....	2
DAFTAR ISI.....	3
BAB I	
DESKRIPSI TUGAS.....	4
1.1 Spesifikasi Wajib.....	5
BAB II	
LANDASAN TEORI.....	9
2.1 Pattern Matching.....	9
2.2 Knuth-Morris-Pratt (KMP).....	9
2.3 Boyer-Moore.....	10
2.4 Regular Expression.....	10
2.5 Weighted Levenshtein Distance.....	10
2.6 Aho-Corasick.....	11
2.7 Rabin-Karp.....	11
2.8 Preprocessing dan Normalization Teks.....	12
2.9 Chromium Browser Extension.....	12
2.10 Arsitektur Extension.....	12
BAB III	
ANALISIS PEMECAHAN MASALAH.....	13
3.1 Langkah-Langkah Pemecahan Masalah.....	13
3.2 Pengambilan Teks dari DOM.....	13
3.3 Preprocessing dan Normalization.....	14
3.4 Pemetaan Masalah ke Algoritma KMP.....	14
3.5 Pemetaan Masalah ke Algoritma Boyer-Moore.....	15
3.6 Deteksi Pola dengan RegEx.....	15
3.7 Fuzzy Matching dengan Weighted Levenshtein Distance.....	16
3.8 Arsitektur Extension.....	16
3.9 Alur Kerja Sistem.....	17
3.10 Contoh Ilustrasi Kasus.....	18
BAB IV	
IMPLEMENTASI DAN PENGUJIAN.....	19
4.1 Implementasi.....	19
4.1.1 Arsitektur Ekstensi.....	19
4.1.2 Implementasi Algoritma Pattern Matching.....	19
4.1.3 Implementasi Manipulasi DOM dan UI (Content & Popup).....	20
4.2 Pengujian.....	20
4.2.1 Kasus 1.....	20
4.2.2 Kasus 2.....	20

4.2.3 Kasus 3.....	21
4.2.4 Kasus 4.....	21
4.2.5 Kasus 5.....	22
4.2.6 Analisis Hasil Pengujian.....	22
BAB V	
KESIMPULAN DAN SARAN.....	24
5.1 Kesimpulan.....	24
5.2 Saran.....	24
LAMPIRAN.....	25
DAFTAR PUSTAKA.....	27

BAB I

DESKRIPSI TUGAS

1.1 Spesifikasi Wajib

Pada tugas besar ini, buatlah sebuah *chromium browser extension* yang dapat mendeteksi informasi yang mengandung unsur judul. Sebagai catatan, seperti yang biasa kita temukan dalam website sekarang dengan format <kata><angka>. Contoh: GACOR99, MAXWIN88, MADU308, dan sebagainya. Sistem harus memiliki **minimal fitur** dengan detail seperti berikut.

1. Extension yang dibuat pada tugas besar ini dikembangkan dengan bahasa **Typescript**. Dalam proses pencocokan kata kunci, sistem wajib mengimplementasikan algoritma **Knuth-Morris-Pratt** dan **Boyer Moore** serta **RegEx**. Untuk algoritma selain RegEx, pencarian dilakukan menggunakan file [keyword.txt](#) (file ini boleh dimodifikasi sesuai kebutuhan) yang dipisahkan dengan endline untuk melakukan string matching secara iterative, sehingga pengguna tidak memasukkan input, tapi sistem yang membaca file tersebut secara langsung.

Untuk pencarian dengan regex, format pencarian **seminimal mungkin** dapat handle 2 atau 3 angka di belakang kata tanpa spasi. Contoh: MAXWIN234, SLOT99. Test case yang diberikan nanti akan sangat bervariasi, sehingga pastikan pencarian RegEx diimplementasikan dengan detail dan dapat **handle banyak edge case**. Adapun juga algoritma *Aho-Corasick* dan *Rabin Karp* apabila ingin mengerjakan bonus, sesuai tertera di Spesifikasi Bonus dibawah.

2. Perlu diketahui bahwa, Developer Judol pun juga membuat penamaan dari websitenya, seperti pada teknik *Phising*, jadi membuat nama domain yang serupa dengan pada umumnya, namun nyatanya berbeda. Ini terjadi oleh karena adanya perbedaan kode komputer (*unicode*), dan komputer mengiranya adalah karakter yang berbeda. Contoh:

Gacor999	Gacor999
HOKI88	H0KI88
dan lain sebagainya	

Maka dari itu, sistem harus bisa membaca kecocokan secara *exact matching* dan juga *fuzzy matching* (apabila tidak ditemukan kecocokan yang persis). Untuk setiap kata kunci yang tidak memiliki kemunculan sama sekali pada saat *exact matching*, lakukan pencarian kembali dengan perhitungan tingkat kemiripan menggunakan algoritma [Weighted Levenshtein Distance](#). Penyesuaian bobot (*weight*) ini krusial karena pelaku sering kali menyasiasi strategi dengan menggunakan huruf pengganti yang mirip secara visual (misalnya mengubah huruf 'O' menjadi angka '0', 'A' menjadi '4', atau 'I' menjadi '1'). Implementasi harus memperhatikan hal-hal berikut:

1. **Weighted Cost:** Berikan bobot penalti yang lebih kecil untuk substitusi karakter yang mirip secara visual dibandingkan dengan karakter yang benar-benar berbeda.
2. **Thresholding:** Tentukan nilai ambang batas (*threshold*) kemiripan yang tepat untuk memastikan sistem tetap memberikan hasil yang relevan namun tetap menghindari munculnya *false positive*.

Algoritma ini memungkinkan sistem untuk tetap menampilkan hasil pencarian yang relevan meskipun terdapat perbedaan minor, kesalahan ketik, atau upaya manipulasi teks oleh pengguna. *Perlu diperhatikan bahwa secara konsep, exact matching juga merupakan bagian dari fuzzy matching.*

$$\text{lev}(a, b) = \begin{cases} |a| & \text{if } |b| = 0, \\ |b| & \text{if } |a| = 0, \\ \text{lev}(\text{tail}(a), \text{tail}(b)) & \text{if } \text{head}(a) = \text{head}(b), \\ 1 + \min \begin{cases} \text{lev}(\text{tail}(a), b) \\ \text{lev}(a, \text{tail}(b)) \\ \text{lev}(\text{tail}(a), \text{tail}(b)) \end{cases} & \text{otherwise} \end{cases}$$

Weighted Levenshtein Distance Formula

3. Jadi sistem menjalankan pengecekan dengan hirarki sebagai berikut:
 - a. **Exact Matching:** Algoritma *pattern matching* untuk mencari kata kunci dasar dari keyword.txt

- b. RegEx Matching: Secara paralel atau sekuensial, jalankan Regex untuk menangkap pola <kata><angka>
 - c. Fuzzy Matching: Kalau string dalam DOM tidak lolos namun memiliki kemiripan, jalankan dengan Weighted Levenshtein Distance dalam upaya manipulasi karakter
- 4. Apabila ditemukan result keyword, maka elemen DOM terkait harus ditandai, dengan warna bebas. Namun highlight tidak boleh merusak layout, tepat sasaran dan dapat terhapus saat rescanning.
- 5. Saat user hover pada elemen yang terdeteksi, maka tampilkan tooltip dengan informasi berikut:
 - a. Keyword yang terdeteksi
 - b. Algoritma yang digunakan
 - c. Jumlah kemunculan
 - d. Waktu eksekusi

Tooltip dibuat dengan custom DOM element.

- 6. Terdapat juga statistik secara realtime pada popup extension yang menampilkan:
 - a. Total keyword yang ditemukan
 - b. Perbandingan jumlah keyword yang satu dengan lainnya, menggunakan visualisasi
 - c. Waktu eksekusi tiap algoritma
 - d. Jumlah match tiap algoritma
- 7. Setiap kelompok harap mengisi nama kelompok dan anggotanya pada link [berikut](#), paling lambat **Minggu, 17 Mei 2026 23.59**. Nama kelompok dilarang mengandung unsur SARA, penghinaan, provokasi, atau hal-hal lain yang bertentangan dengan norma hukum.

Perlu diperhatikan bahwa kelompok tubes untuk mata kuliah ini **tidak boleh sama**. Jadi jika berkelompok dengan X pada tubes ini, maka pada tubes selanjutnya anda tidak diperkenankan berkelompok dengan X.

Perhatian

- a. Seluruh algoritma string matching wajib dibuat *from scratch*
- b. **Dilarang keras** menggunakan
 - i. *string.includes()*
 - ii. *string.indexOf()*
 - iii. *Built-in search function*
 - iv. *External string matching library*
- c. Implementasi manual wajib mencakup:
 - i. *Border function*
 - ii. *Failure function*
 - iii. *Last occurrence table*
 - iv. *Shifting process*
 - v. *Comparison Counting*
- d. Regex **diperbolehkan** menggunakan regex engine dari JavaScript

BAB II

LANDASAN TEORI

2.1 Pattern Matching

Pattern matching adalah proses mencari kemunculan suatu pola atau pattern di dalam teks. Dalam tugas ini, pattern matching digunakan untuk mendeteksi kata kunci dan pola teks yang berhubungan dengan judi online pada halaman web. Teks yang diperiksa berasal dari DOM halaman web, sedangkan pattern berasal dari daftar keyword pada file keywords.txt maupun pola RegEx.

Masalah pattern matching secara umum memiliki dua komponen utama, yaitu teks dan pattern. Teks adalah string panjang tempat pencarian dilakukan, sedangkan pattern adalah string yang ingin dicari. Pada proyek ini, teks berupa konten halaman web, sedangkan pattern berupa keyword seperti GACOR, MAXWIN, SLOT, dan kata lain yang berpotensi berkaitan dengan judi online.

2.2 Knuth-Morris-Pratt (KMP)

Knuth-Morris-Pratt atau KMP adalah algoritma pencarian string yang memanfaatkan informasi dari pattern untuk menghindari perbandingan ulang yang tidak perlu. Algoritma ini membangun tabel border atau failure function sebelum proses pencarian dilakukan.

Tabel border menunjukkan panjang prefix dari pattern yang juga merupakan suffix sampai posisi tertentu. Ketika terjadi mismatch antara karakter pattern dan karakter teks, algoritma tidak perlu kembali ke awal pattern, tetapi dapat menggunakan tabel border untuk menentukan posisi perbandingan berikutnya.

Secara umum, tahapan algoritma KMP adalah:

1. Membangun border function atau failure function dari pattern.
2. Membandingkan pattern dengan teks dari kiri ke kanan.
3. Jika karakter cocok, indeks teks dan indeks pattern maju.
4. Jika terjadi mismatch, indeks pattern digeser berdasarkan failure function.
5. Jika seluruh pattern cocok, posisi kemunculan pattern dicatat.

Kelebihan KMP adalah kompleksitas waktunya linear terhadap panjang teks dan pattern, yaitu $O(n + m)$, dengan n sebagai panjang teks dan m sebagai panjang pattern. Pada proyek ini, KMP digunakan untuk exact matching terhadap setiap keyword yang dibaca dari keywords.txt.

2.3 Boyer-Moore

Boyer-Moore adalah algoritma pencarian string yang melakukan perbandingan pattern dari kanan ke kiri. Algoritma ini dikenal efisien karena dapat melakukan pergeseran pattern lebih jauh ketika terjadi mismatch.

Salah satu komponen penting pada Boyer-Moore adalah last occurrence table. Tabel ini menyimpan posisi kemunculan terakhir setiap karakter pada pattern. Ketika terjadi mismatch, algoritma menggunakan informasi tersebut untuk menentukan seberapa jauh pattern dapat digeser.

Secara umum, tahapan algoritma Boyer-Moore adalah:

1. Membangun last occurrence table dari pattern.
2. Menempatkan pattern sejajar dengan bagian teks yang sedang diperiksa.
3. Membandingkan karakter pattern dan teks dari kanan ke kiri.
4. Jika cocok seluruhnya, posisi match dicatat.
5. Jika terjadi mismatch, pattern digeser berdasarkan last occurrence table.

Pada kasus tertentu, Boyer-Moore dapat bekerja sangat cepat karena tidak selalu memeriksa setiap karakter teks. Pada proyek ini, Boyer-Moore digunakan untuk exact matching terhadap keyword dari keywords.txt dan hasilnya dibandingkan dengan algoritma lain berdasarkan jumlah match, jumlah perbandingan, dan waktu eksekusi.

2.4 Regular Expression

Regular Expression atau RegEx adalah metode pencarian teks berdasarkan pola tertentu. RegEx tidak digunakan untuk membaca keyword dari file, tetapi digunakan untuk menangkap pola umum yang sering muncul pada konten judi online.

Pada tugas ini, RegEx digunakan untuk mendeteksi format kata yang diikuti angka tanpa spasi, seperti MAXWIN88, GACOR99, SLOT234, dan bentuk sejenis. Pola ini penting karena banyak konten judi online menggunakan kombinasi kata dan angka untuk membuat nama brand, situs, atau promosi.

2.5 Weighted Levenshtein Distance

Weighted Levenshtein Distance adalah pengembangan dari Levenshtein Distance yang menghitung jarak antara dua string berdasarkan operasi insert, delete, dan substitute. Berbeda dengan Levenshtein biasa, Weighted Levenshtein memberikan bobot berbeda untuk operasi tertentu.

Pada tugas ini, Weighted Levenshtein digunakan untuk fuzzy matching. Fuzzy matching diperlukan karena konten judi online dapat memanipulasi penulisan keyword agar tidak terdeteksi secara exact. Contohnya adalah mengganti huruf O dengan angka 0, huruf A dengan angka 4, atau huruf I dengan angka 1.

Substitusi karakter yang mirip secara visual diberi penalti lebih kecil dibanding substitusi karakter yang tidak mirip. Setelah jarak dihitung, sistem menentukan tingkat kemiripan antara token pada halaman dan keyword. Jika tingkat kemiripan melewati threshold tertentu, token dianggap sebagai hasil fuzzy matching.

2.6 Aho-Corasick

Algoritma Aho-Corasick adalah algoritma pencocokan string yang dirancang khusus untuk menangani pencarian banyak pola (*multiple pattern matching*) secara serentak dalam satu kali pembacaan teks target. Pendekatan utama algoritma ini adalah membangun sebuah mesin *Finite State Automaton* (FSA) berupa struktur data *Trie* (pohon awalan) dari himpunan kata kunci yang ada. Mesin ini beroperasi menggunakan tiga fungsi utama: fungsi transisi (*Go-To*) untuk menelusuri *node* berdasarkan karakter yang cocok, fungsi kegagalan (*Failure*) untuk menentukan *state* yang dituju saat terjadi *mismatch* berdasarkan *longest proper suffix*, dan fungsi keluaran (*Output*) untuk mengumpulkan kata kunci yang berhasil ditemukan pada jalur *state* tersebut.

Berbeda dengan algoritma pencocokan tunggal (seperti KMP atau Boyer-Moore) yang harus dieksekusi berulang kali sebanyak jumlah pola, Aho-Corasick sangat efisien karena memproses teks target tepat satu kali. Proses pembangunan *Trie* memiliki kompleksitas waktu pembentukan $O(M)$, di mana M adalah total panjang seluruh kata kunci. Sementara itu, waktu pencariannya memiliki kompleksitas $O(N + Z)$, dengan N merepresentasikan panjang teks target dan Z adalah jumlah kemunculan pola. Efisiensi inilah yang membuatnya sangat ideal untuk mengevaluasi sebuah web terhadap ratusan *keyword* sekaligus secara bersamaan.

2.7 Rabin-Karp

Rabin-Karp adalah algoritma pencocokan string yang memanfaatkan fungsi *hash* untuk menemukan keberadaan suatu pola (*pattern*) di dalam teks. Alih-alih melakukan perbandingan per karakter secara langsung, algoritma ini membandingkan nilai *hash* dari pola dengan nilai *hash* dari *window* teks yang sedang dievaluasi. Untuk mempercepat pergeseran evaluasi dari kiri ke kanan, Rabin-Karp menggunakan teknik *Rolling Hash*, yang memungkinkan penghitungan *hash* untuk *window* teks berikutnya dilakukan dalam waktu konstan $O(1)$ dengan cara mengurangi bobot karakter pertama yang keluar dan menambahkan bobot karakter baru yang masuk. Verifikasi string secara mendetail (karakter demi karakter) hanya akan dilakukan jika kedua nilai *hash* cocok, guna menghindari kesalahan identifikasi akibat kolisi *hash* (*spurious hit*).

Secara matematis, perhitungan *hash* pada algoritma ini menggunakan formula polinomial dengan operasi modulo, untuk memastikan nilai *hash* tetap berada dalam batas tipe data *integer* dan menekan rasio kolisi. Rabin-Karp memiliki kompleksitas waktu rata-rata dan pencapaian terbaik (*best-case*) sebesar $O(N + M)$, di mana N adalah panjang teks dan M adalah panjang pola. Meskipun demikian, pada kasus terburuk (*worst-case*) di mana terdapat terlalu banyak *spurious hit* karena fungsi *hash* yang kurang optimal, kompleksitas algoritma ini dapat merosot menjadi $O(N \times M)$ karena sistem harus memverifikasi *string* secara manual di hampir setiap pergeseran *window*.

2.8 Preprocessing dan Normalization Teks

Preprocessing adalah proses menyiapkan data sebelum diproses oleh algoritma. Pada proyek ini, preprocessing dilakukan pada keyword dan teks halaman web. Keyword dari keywords.txt dibaca per baris, dihapus spasi kosongnya, diubah ke bentuk uppercase, dan duplikat dihilangkan.

Normalization dilakukan agar perbandingan teks lebih konsisten. Misalnya, pencarian exact matching dilakukan secara case-insensitive sehingga perbedaan huruf besar dan kecil tidak memengaruhi hasil. Untuk fuzzy matching, normalization juga membantu membandingkan karakter yang memiliki bentuk visual mirip.

2.9 Chromium Browser Extension

Chromium browser extension adalah program tambahan yang berjalan pada browser berbasis Chromium, seperti Google Chrome, Microsoft Edge, Brave, dan Opera. Extension digunakan untuk menambah fungsi browser tanpa mengubah kode asli browser.

Pada proyek ini, extension digunakan untuk membaca teks pada halaman web dan mendeteksi konten yang berpotensi mengandung unsur judi online. Extension dibangun menggunakan TypeScript dan menggunakan Manifest V3 sebagai format konfigurasi.

2.10 Arsitektur Extension

Extension yang dibangun terdiri dari beberapa komponen utama, yaitu manifest, content script, popup, file keyword, dan build output.

Manifest adalah file konfigurasi utama yang dibaca browser saat extension dimuat. File ini menentukan permission, popup, content script, dan resource yang dapat diakses.

Content script adalah script yang berjalan pada halaman web. Pada proyek ini, content script digunakan untuk mengambil teks dari DOM, menjalankan pipeline deteksi, menerapkan highlight, dan menyimpan hasil scan.

Popup adalah antarmuka yang muncul ketika pengguna menekan icon extension. Popup digunakan untuk menampilkan statistik hasil deteksi, seperti total temuan, jumlah match per algoritma, waktu eksekusi, perbandingan keyword, tombol Rescan, dan toggle blur.

File keywords.txt digunakan sebagai sumber keyword untuk exact matching. Folder dist berisi hasil build extension yang dapat dimuat ke browser melalui fitur Load unpacked.

BAB III

ANALISIS PEMECAHAN MASALAH

3.1 Langkah-Langkah Pemecahan Masalah

Permasalahan utama pada tugas ini adalah bagaimana mendeteksi teks pada halaman web yang berpotensi mengandung unsur judi online. Teks tersebut dapat berupa keyword biasa, kombinasi kata dan angka, maupun kata yang dimanipulasi dengan karakter yang mirip secara visual.

Langkah-langkah pemecahan masalah yang dilakukan adalah sebagai berikut:

1. Membuat browser extension berbasis Chromium agar sistem dapat berjalan langsung pada halaman web.
2. Mengambil teks dari halaman web melalui DOM menggunakan content script.
3. Membaca daftar keyword dari file keywords.txt.
4. Melakukan preprocessing dan normalization pada keyword dan teks.
5. Menjalankan exact matching menggunakan algoritma KMP dan Boyer-Moore.
6. Menjalankan RegEx matching untuk mendeteksi pola kata yang diikuti angka.
7. Menjalankan fuzzy matching menggunakan Weighted Levenshtein Distance untuk keyword yang tidak ditemukan secara exact.
8. Menggabungkan hasil deteksi dari seluruh algoritma.
9. Memberi highlight pada teks yang terdeteksi.
10. Menampilkan statistik hasil deteksi pada popup extension.

Dengan langkah tersebut, sistem tidak hanya mendeteksi keyword yang sama persis, tetapi juga dapat mendeteksi pola umum dan variasi teks yang dimanipulasi.

3.2 Pengambilan Teks dari DOM

Teks halaman web diambil menggunakan content script yang berjalan pada halaman yang dibuka pengguna. Content script membaca struktur DOM halaman dan mengambil text node yang berisi teks.

Sistem tidak membaca file HTML mentah secara langsung, melainkan membaca DOM yang sudah diproses oleh browser. Hal ini penting karena halaman web modern sering kali menghasilkan konten secara dinamis menggunakan JavaScript. Dengan membaca DOM, extension dapat mengambil teks yang benar-benar muncul pada halaman.

Beberapa elemen tidak diproses karena bukan bagian utama dari konten halaman, seperti script, style, noscript, input, textarea, select, dan option. Elemen-elemen tersebut dilewati agar sistem tidak mendeteksi teks dari kode program, styling, atau input form.

Setiap text node yang dibaca disimpan sebagai target dengan informasi teks, posisi awal, posisi akhir, dan referensi node DOM. Informasi posisi tersebut digunakan untuk memetakan hasil deteksi kembali ke bagian teks yang sesuai pada halaman.

3.3 Preprocessing dan Normalization

Preprocessing dilakukan untuk menyiapkan keyword dan teks sebelum diproses oleh algoritma. Keyword dibaca dari file keywords.txt, kemudian dipisahkan berdasarkan baris. Setiap keyword dihapus spasi kosong di awal dan akhir, diubah menjadi uppercase, dan duplikat dihilangkan.

Normalization dilakukan agar proses pencocokan lebih konsisten. Pada exact matching, teks dan keyword dibandingkan secara case-insensitive sehingga perbedaan huruf besar dan huruf kecil tidak memengaruhi hasil deteksi. Contohnya, keyword GACOR tetap dapat mendeteksi teks gacor, Gacor, atau GACOR.

Pada fuzzy matching, normalization juga membantu sistem membandingkan token halaman dengan keyword meskipun terdapat variasi karakter. Contohnya, H0KI dapat dibandingkan dengan HOKI karena karakter 0 memiliki bentuk visual yang mirip dengan huruf O.

3.4 Pemetaan Masalah ke Algoritma KMP

Pada algoritma KMP, keyword dari keywords.txt diperlakukan sebagai pattern, sedangkan teks halaman web diperlakukan sebagai text. Karena terdapat banyak keyword, proses pencarian dilakukan secara iteratif untuk setiap keyword.

Pemetaan elemen masalah ke KMP adalah sebagai berikut:

- Text: gabungan teks yang diambil dari DOM halaman web.
- Pattern: satu keyword dari keywords.txt.
- Border/failure function: tabel yang dibangun dari keyword untuk menentukan pergeseran saat terjadi mismatch.
- Match result: posisi awal dan akhir keyword yang ditemukan pada teks.
- Comparison count: jumlah perbandingan karakter selama proses pencarian.

Tahapan pencarian menggunakan KMP dimulai dengan membangun border function untuk pattern. Setelah itu, pattern dibandingkan dengan teks dari kiri ke kanan. Jika karakter cocok, indeks teks dan indeks pattern bergerak maju. Jika terjadi mismatch, indeks pattern digeser berdasarkan border function tanpa perlu mengulang perbandingan dari awal.

Dengan pemetaan ini, KMP digunakan untuk mencari keyword secara exact dan efisien pada teks halaman web.

3.5 Pemetaan Masalah ke Algoritma Boyer-Moore

Pada algoritma Boyer-Moore, keyword juga diperlakukan sebagai pattern dan teks halaman web sebagai text. Perbedaananya, Boyer-Moore melakukan perbandingan dari kanan ke kiri pada pattern.

Pemetaan elemen masalah ke Boyer-Moore adalah sebagai berikut:

- Text: gabungan teks dari DOM halaman web.
- Pattern: satu keyword dari keywords.txt.
- Last occurrence table: tabel posisi terakhir setiap karakter pada pattern.
- Shifting process: proses menggeser pattern berdasarkan karakter yang menyebabkan mismatch.
- Match result: posisi awal dan akhir keyword yang ditemukan.
- Comparison count: jumlah perbandingan karakter selama pencarian.

Tahapan Boyer-Moore dimulai dengan membangun last occurrence table dari keyword. Setelah itu, pattern disejajarkan dengan bagian teks yang sedang diperiksa. Perbandingan dilakukan dari karakter paling kanan pattern. Jika terjadi mismatch, pattern digeser berdasarkan posisi terakhir karakter mismatch pada pattern.

Dengan strategi ini, Boyer-Moore dapat melewati beberapa karakter sekaligus dan berpotensi lebih cepat pada teks yang panjang.

3.6 Deteksi Pola dengan RegEx

Selain keyword dari file, sistem juga perlu mendeteksi pola kata yang diikuti angka. Pola ini sering muncul pada konten judi online, misalnya MAXWIN88, GACOR99, SLOT234, dan bentuk sejenis.

RegEx digunakan untuk mendeteksi pola tersebut karena formatnya tidak selalu berasal dari keyword tertentu. RegEx mencari token yang memiliki susunan kata dan angka tanpa spasi. Dengan pendekatan ini, sistem dapat mendeteksi variasi brand atau promosi yang belum tentu ada di keywords.txt.

Contoh pemetaan masalah ke RegEx:

- Text: gabungan teks dari DOM halaman web.
- Pattern: aturan RegEx untuk kata yang diikuti 2 sampai 4 angka.

- Match result: token yang sesuai dengan pola.
- Keyword output: bagian kata dari token yang ditemukan.
- Algorithm label: regex.

RegEx dijalankan bersama algoritma lain dan hasilnya digabungkan ke dalam hasil scan.

3.7 Fuzzy Matching dengan Weighted Levenshtein Distance

Fuzzy matching digunakan ketika keyword tidak ditemukan secara exact. Hal ini diperlukan karena teks judi online dapat dimanipulasi agar tidak terdeteksi secara langsung. Contohnya adalah mengganti O menjadi 0, A menjadi 4, I menjadi 1, atau menggunakan karakter Unicode yang terlihat mirip.

Pada Weighted Levenshtein Distance, sistem menghitung jarak antara token dari halaman dan keyword. Operasi substitusi karakter yang mirip secara visual diberi bobot lebih kecil dibanding substitusi karakter yang berbeda jauh.

Pemetaan elemen masalah ke Weighted Levenshtein Distance adalah sebagai berikut:

- Token: potongan teks dari halaman web.
- Keyword: kata kunci yang belum ditemukan oleh exact matching.
- Weighted cost: bobot operasi edit, terutama substitusi karakter mirip.
- Distance: nilai jarak antara token dan keyword.
- Similarity score: tingkat kemiripan token dengan keyword.
- Threshold: batas minimal kemiripan agar token dianggap cocok.

Jika similarity score melewati threshold, token dianggap sebagai hasil fuzzy matching. Dengan cara ini, sistem dapat mendeteksi variasi seperti H0KI88 yang mirip dengan HOKI.

3.8 Arsitektur Extension

Arsitektur extension dibagi menjadi beberapa modul agar setiap bagian memiliki tanggung jawab yang jelas.

Modul utama sistem adalah sebagai berikut:

1. Manifest

Manifest berada pada file manifest.json. File ini dibaca oleh browser saat extension dimuat. Manifest menentukan content script, popup, permission, host permission, dan resource yang dapat diakses.

2. Content Script
Content script berjalan pada halaman web. Tugasnya adalah mengambil teks dari DOM, memanggil pipeline deteksi, menerapkan highlight, dan menyimpan hasil scan.
3. Keyword Loader
Keyword loader membaca file keywords.txt dari extension package. File tersebut diproses menjadi array keyword yang digunakan oleh KMP, Boyer-Moore, dan fuzzy matching.
4. Detection Pipeline
Detection pipeline mengatur urutan pemanggilan algoritma. Pipeline menjalankan KMP, Boyer-Moore, RegEx, dan Weighted Levenshtein, lalu menggabungkan hasilnya menjadi satu ScanResult.
5. Algorithm Module
Algorithm module berisi implementasi algoritma KMP, Boyer-Moore, RegEx, dan Weighted Levenshtein Distance. Setiap algoritma mengembalikan hasil dalam format yang sama agar dapat digabungkan.
6. Highlighter
Highlighter bertugas memetakan hasil deteksi ke DOM dan membungkus teks yang terdeteksi dengan elemen span. Elemen ini digunakan untuk memberi highlight dan tooltip.
7. Popup
Popup adalah antarmuka pengguna extension. Popup menampilkan total temuan, statistik tiap algoritma, perbandingan jumlah keyword, tombol Rescan, dan toggle blur.
8. Shared Contract
Shared contract berisi tipe data yang digunakan bersama antar modul, seperti TextMatch, AlgorithmResult, AlgorithmRunStats, dan ScanResult. Dengan kontrak ini, setiap modul memiliki format data yang konsisten.

3.9 Alur Kerja Sistem

Alur kerja sistem dimulai ketika pengguna membuka halaman web. Browser menjalankan content script berdasarkan konfigurasi pada manifest. Content script kemudian memuat keyword, mengambil teks dari DOM, dan menjalankan pipeline deteksi.

Setelah pipeline selesai, hasil deteksi digunakan untuk memberi highlight pada teks halaman. Hasil scan juga disimpan agar popup dapat menampilkan statistik. Ketika pengguna menekan tombol Rescan, content script menjalankan ulang proses scan secara manual.

Alur sistem secara ringkas adalah sebagai berikut:

Halaman web dibuka -> content script berjalan -> keyword dimuat -> teks DOM dikumpulkan -> algoritma dijalankan -> hasil deteksi digabungkan -> highlight diterapkan -> hasil scan disimpan -> popup menampilkan statistik.

3.10 Contoh Ilustrasi Kasus

Misalkan sebuah halaman web memiliki teks berikut:

“Promo MAXWIN88 dan H0KI88 tersedia hari ini.”

Keyword pada keywords.txt berisi:

- MAXWIN
- HOKI
- GACOR

Proses deteksi berjalan sebagai berikut:

1. Content script mengambil teks dari DOM.
2. Keyword MAXWIN, HOKI, dan GACOR dibaca dari keywords.txt.
3. KMP dan Boyer-Moore melakukan exact matching terhadap keyword.
4. RegEx mendeteksi MAXWIN88 sebagai pola kata yang diikuti angka.
5. Jika HOKI tidak ditemukan secara exact karena teks menggunakan H0KI, Weighted Levenshtein menghitung kemiripan antara H0KI dan HOKI.
6. Karena O dan 0 mirip secara visual, bobot substitusi lebih kecil dan similarity score dapat melewati threshold.
7. Sistem mencatat hasil deteksi.
8. Teks MAXWIN88 dan H0KI88 diberi highlight pada halaman.
9. Popup menampilkan total deteksi, algoritma yang digunakan, dan waktu eksekusi.

Dengan contoh tersebut, sistem dapat menangani dua jenis kasus sekaligus, yaitu pola kata-angka dan variasi karakter yang dimanipulasi.

BAB IV

IMPLEMENTASI DAN PENGUJIAN

4.1 Implementasi

Sistem ekstensi dibangun dengan arsitektur berbasis *Content Script* dan *Popup*, di mana logika pemrosesan dan algoritma pencarian string dipisahkan ke dalam modul-modul (*separation of concerns*) agar mudah dipelihara dan diuji.

4.1.1 Arsitektur Ekstensi

Ekstensi ini dipecah menjadi beberapa bagian utama:

1. Popup Script (src/popup/index.ts): Bertanggung jawab untuk menampilkan antarmuka pengguna (UI) secara *real-time* ketika ikon ekstensi diklik. Bagian ini menampilkan statistik (waktu eksekusi, jumlah kata yang cocok, jumlah *keyword*).
2. Content Script (src/content/index.ts): Merupakan modul yang disuntikkan langsung ke dalam halaman web yang sedang dibuka. Modul ini bertugas mengekstraksi teks dari *Document Object Model* (DOM), memanggil algoritma pencarian, dan melakukan manipulasi DOM (seperti *highlighting* atau *blur*).
3. Keyword Loader (src/keywords/loadKeywords.ts): Bertugas memuat senarai *keyword* judi online dari berkas teks (keywords.txt) secara sekuensial untuk digunakan sebagai basis pencocokan algoritma.

```
export function parseKeywords(source: string): string[] {
  const keywords: string[] = [];
  const seen = new Set<string>();

  for (const rawLine of source.split(/\r?\n/)) {
    const keyword = rawLine.trim();
    if (keyword.length === 0) {
      continue;
    }

    const normalized = keyword.toUpperCase();
    if (seen.has(normalized)) {
      continue;
    }

    seen.add(normalized);
    keywords.push(normalized);
  }

  return keywords;
}
```

4.1.2 Implementasi Algoritma Pattern Matching

Algoritma pencocokan pola diimplementasikan di dalam direktori `src/algorithms/`. Ekstensi mendukung pencarian dengan pendekatan *exact match*, *fuzzy match*, serta pendekatan optimasi lainnya (termasuk fitur bonus):

1. Knuth-Morris-Pratt (KMP) (`kmp.ts`): Diimplementasikan dengan membangun tabel fungsi pinggiran (*border function* / *LPS array*) terlebih dahulu. Algoritma ini memastikan tidak ada karakter pada teks yang dievaluasi berulang kali saat terjadi ketidakcocokan (*mismatch*).

```
export function runKmp(input: MatcherInput): AlgorithmResult {
  const startMs = performance.now();
  const matches: TextMatch[] = [];
  let totalComparisons = 0;

  // Ubah teks ke uppercase untuk memastikan case-insensitive matching
  const text = input.text.toUpperCase();
  const n = text.length;

  for (const keyword of input.keywords) {
    if (keyword.length === 0) continue;

    const pattern = keyword.toUpperCase();
    const m = pattern.length;

    const metrics = { comparisons: 0 };
    const lps = computeBorderFunction(pattern, metrics);
    let keywordComparisons = metrics.comparisons;

    let i = 0; // index untuk text
    let j = 0; // index untuk pattern

    while (i < n) {
      keywordComparisons++; // Menghitung proses komparasi text[i] dengan pattern[j]
      if (pattern[j] === text[i]) {
        i++;
        j++;
      }
    }
  }
}
```

2. Boyer-Moore (BM) (`boyerMoore.ts`): Diimplementasikan dengan memanfaatkan *bad character heuristic*. Algoritma akan membuat tabel kemunculan terakhir (*last occurrence*) untuk menggeser pola sejauh mungkin ke kanan bila terjadi ketidakcocokan, mengevaluasi string dari kanan ke kiri.
3. Regular Expression (`regexMatcher.ts`): Menangani berbagai *edge case* di mana *keyword* disamarkan menggunakan angka, simbol, atau karakter yang mirip (contoh: "sl0t", "g4c0r", "sl@t").


```
export function runRegexMatcher(input: MatcherInput): AlgorithmResult {
  const startMs = performance.now();
  const matches: TextMatch[] = [];
  let totalComparisons = 0;

  const text = input.text;

  // Kumpulan regex dengan 2 capture group: (kata)(angka)
  const patterns: RegExp[] = [
    // Pattern 1: kata >= 3 huruf diikuti 2-4 angka
    /\b([a-zA-Z]{3,})(\d{2,4})\b/g,

    // Pattern 2: kata campur angka di dalamnya
    /\b([a-zA-Z][a-zA-Z0-9]*[a-zA-Z])(\d{2,4})\b/g,

    // Pattern 3: kata 2 huruf diikuti >= 3 angka
    /\b([a-zA-Z]{2})(\d{3,4})\b/g,
  ];
}
```

4. Weighted Levenshtein (weightedLevenshtein.ts): Digunakan untuk *fuzzy matching* dengan menghitung jarak edit antara kata di teks dengan daftar *keyword*. Bobot diberikan secara khusus untuk pengantian karakter yang memiliki kedekatan visual atau fonetik.
5. Aho-Corasick (Bonus - ahoCorasick.ts): Mengimplementasikan *Trie* dan *failure links*. Sangat efisien untuk mencari banyak *keyword* judi online sekaligus dalam satu iterasi melintasi dokumen web.
6. Rabin-Karp (Bonus - rabinKarp.ts): Pencocokan pola menggunakan fungsi *rolling hash*. Digunakan untuk mendeteksi *keyword* dengan membandingkan nilai *hash* pola dengan *hash* teks bagian per bagian.

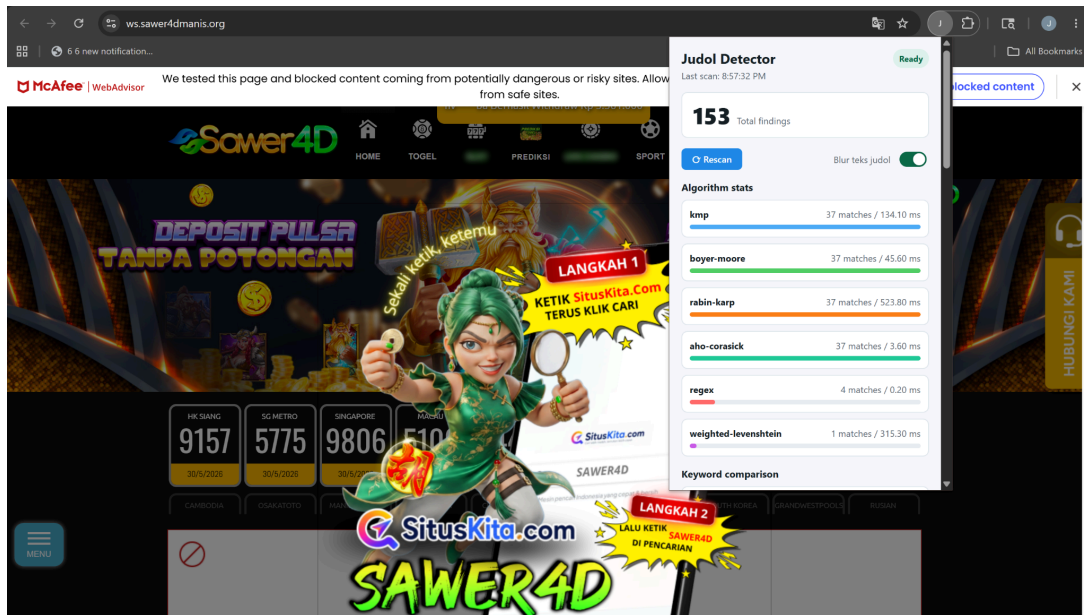
4.1.3 Implementasi Manipulasi DOM dan UI (Content & Popup)

1. Highlighting & Tooltip (src/content/highlighting.ts): Saat *keyword* ditemukan oleh algoritma, skrip akan membungkus elemen teks tersebut dengan tag HTML (seperti `<span>`) yang memiliki warna tertentu (merah/kuning). Sebuah *event listener* ditambahkan agar saat kursor mengarah (*hover*) ke kata tersebut, muncul elemen *Tooltip* yang berisi detail algoritma, *keyword* asli, dan waktu eksekusi.
2. Censorship / Blur Teks (Bonus): Selain *highlight*, diterapkan opsi *toggle* untuk melakukan filter dengan menambahkan efek CSS filter: `blur(...)` pada elemen yang mengandung *keyword* terlarang, sehingga teks tidak dapat dibaca oleh pengguna.

4.2 Pengujian

4.2.1 Kasus 1

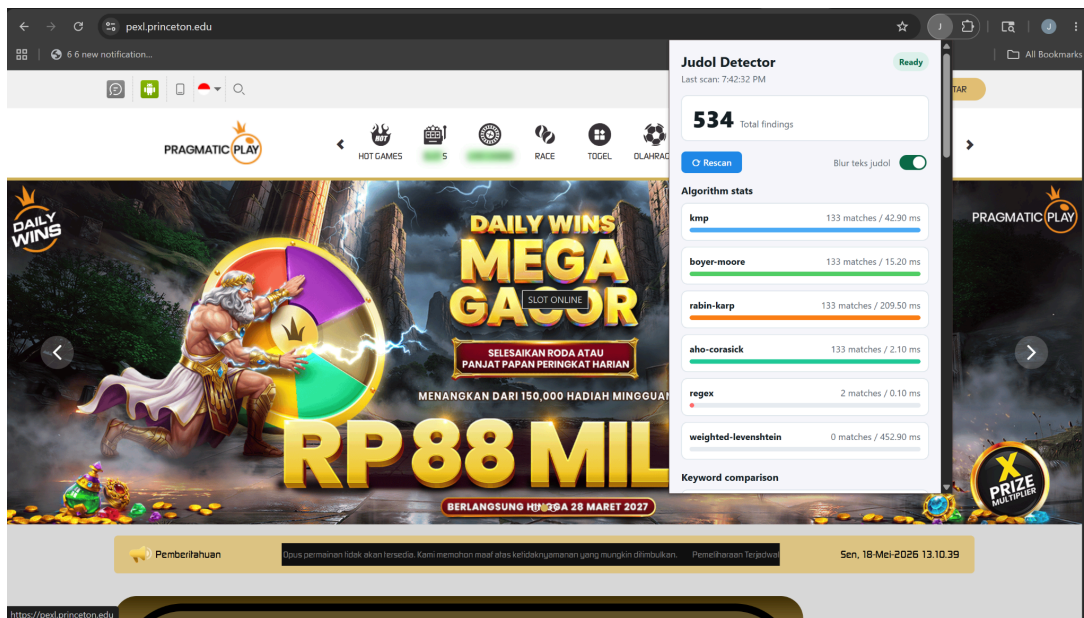
Link: <https://ws.sawer4dmanis.org/>



Gambar 1. Pengujian Kasus 1

4.2.2 Kasus 2

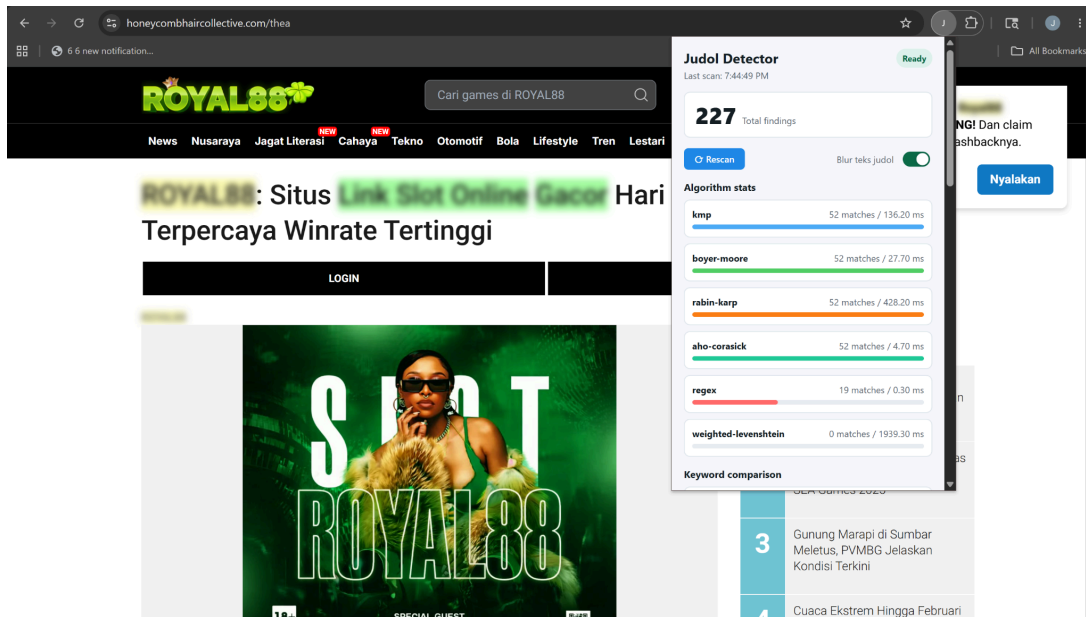
Link: <https://pexl.princeton.edu/>



Gambar 2. Gambar Pengujian Kasus 2

4.2.3 Kasus 3

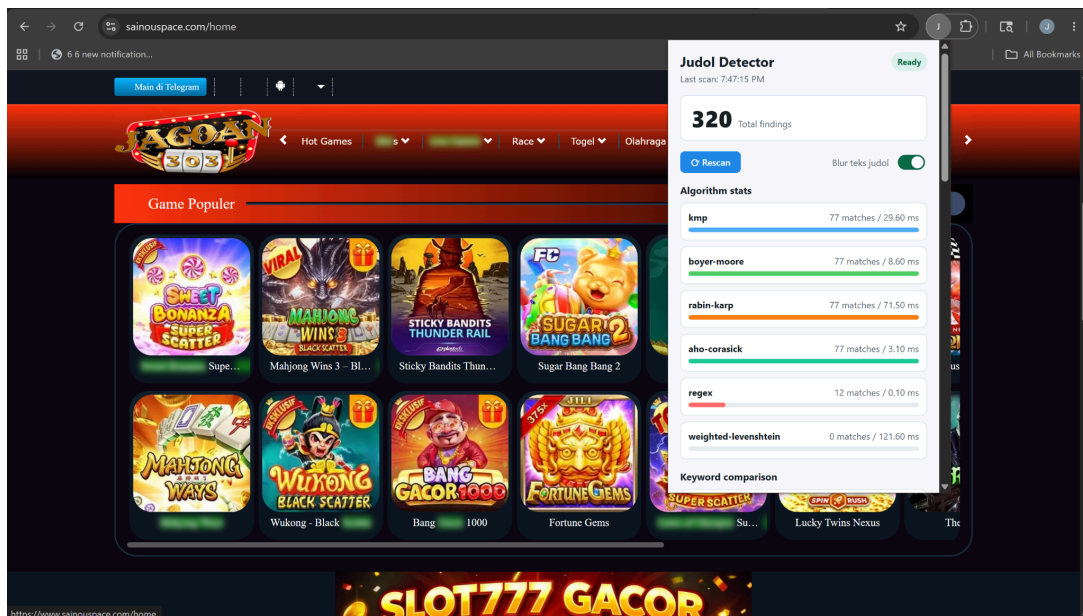
Link: <https://www.honeycombhaircollective.com/thea>



Gambar 3. Gambar Pengujian Kasus 3

4.2.4 Kasus 4

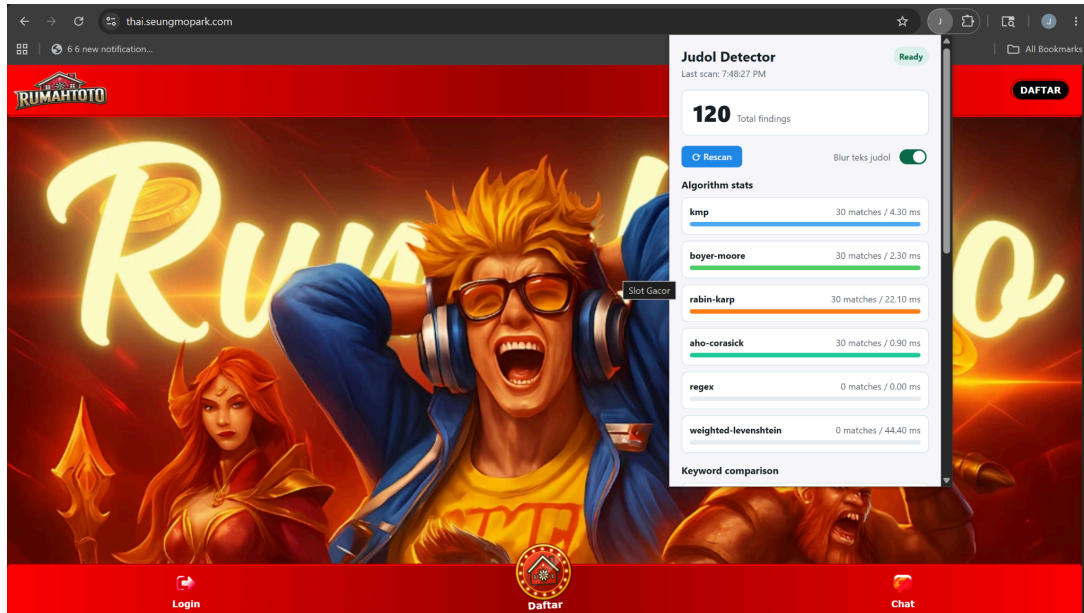
Link: <https://www.sainouspace.com/home>



Gambar 4. Gambar Pengujian Kasus 4

4.2.5 Kasus 5

Link: <https://thai.seungmopark.com/>



Gambar 5. Gambar Pengujian Kasus 5

4.2.6 Analisis Hasil Pengujian

Berdasarkan lima kasus pengujian yang telah dilakukan, didapatkan beberapa analisis utama terkait performa dan karakteristik eksekusi masing-masing algoritma yang diimplementasikan:

1. Performa Exact Matching:

a. Boyer-Moore vs KMP

Secara umum, algoritma Boyer-Moore mencatatkan waktu eksekusi yang lebih cepat dibandingkan KMP. Hal ini dipengaruhi oleh implementasi *Bad Character Heuristic* pada Boyer-Moore yang menggunakan *Last Occurrence Table*. Fitur ini memungkinkan algoritma melakukan lompatan (*shifting*) karakter yang lebih jauh saat terjadi *mismatch* dari kanan ke kiri. Sebaliknya, KMP memproses teks dari kiri ke kanan menggunakan tabel LPS (*Longest Prefix Suffix*), yang pergeserannya cenderung lebih pendek sehingga total komparasinya menjadi lebih banyak.

b. Aho-Corasick

Algoritma ini terbukti menjadi yang paling efisien dalam menangani banyak *keyword* sekaligus. Karena menggunakan struktur data *Trie* yang dilengkapi dengan *Failure Links*, Aho-Corasick hanya perlu memindai teks target tepat satu kali ($O(N)$), terlepas dari seberapa banyak *keyword* yang dicari. Hal ini membuatnya jauh lebih optimal dibanding pencarian iteratif KMP atau Boyer-Moore pada kasus daftar kata kunci yang panjang.

c. Rabin-Karp

Algoritma berbasis *rolling hash* ini memiliki performa di tingkat menengah. Kecepatannya sangat bergantung pada perhitungan fungsi *hash*. Waktu eksekusi algoritma ini dapat sedikit

menurun (*worst-case*) apabila sering terjadi *hash collision* (nilai *hash* sama untuk teks yang berbeda), karena sistem terpaksa melakukan verifikasi kecocokan karakter satu per satu.

2. Performa RegEx:

Penggunaan algoritma Regular Expression (Regex) terbukti sangat cepat dan ringan untuk mendeteksi kata-kata judi yang disamarkan dengan angka. Implementasi Regex membagi fokus pada tiga pola utama. Karena pemrosesan ini berjalan di level string asli tanpa iterasi dari fail teks *keyword*, waktu komputasinya nyaris tidak membebani keseluruhan sistem.

3. Performa Fuzzy Matching:

Weighted Levenshtein Distance (WLD) menjadi penyumbang waktu eksekusi terlama. Tingginya beban waktu ini adalah akibat langsung dari kalkulasi matriks Dynamic Programming berkompleksitas $O(M \times N)$ untuk setiap token. Namun, waktu eksekusi WLD telah berhasil ditekan pada batas yang responsif berkat dua optimasi utama, yaitu dengan mengabaikan evaluasi pada token yang selisih panjangnya melebihi tiga karakter ($MAX_LEN_DIFF = 3$) dan melakukan pemotongan angka di belakang kata (strip trailing digits) sebelum matriks dihitung.

BAB V

KESIMPULAN DAN SARAN

5.1 Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan pengujian yang telah dilakukan, dapat disimpulkan bahwa Judol Detector berhasil dibangun sebagai Chromium browser extension berbasis TypeScript untuk mendeteksi konten yang berpotensi mengandung unsur judi online pada halaman web.

Extension ini mampu membaca teks dari DOM halaman web, memuat daftar keyword dari file keywords.txt, menjalankan algoritma pattern matching, menandai elemen teks yang terdeteksi, serta menampilkan statistik hasil deteksi melalui popup. Dengan demikian, sistem dapat membantu pengguna mengenali konten mencurigakan secara langsung saat membuka halaman web.

Algoritma KMP dan Boyer-Moore digunakan untuk exact matching terhadap keyword yang dibaca dari file keywords.txt. Kedua algoritma tersebut diimplementasikan from scratch tanpa menggunakan built-in search function atau library string matching eksternal. RegEx digunakan untuk mendeteksi pola kata yang diikuti angka, seperti MAXWIN88 atau SLOT99. Weighted Levenshtein Distance digunakan untuk fuzzy matching agar sistem tetap dapat mendeteksi variasi teks yang dimanipulasi, seperti penggantian huruf dengan angka atau karakter yang mirip secara visual.

Dari sisi browser extension, sistem berhasil mengintegrasikan content script, popup, Chrome storage, DOM traversal, highlight, tooltip, dan fitur manual rescans. Hasil deteksi ditampilkan pada halaman melalui highlight dan tooltip, sedangkan statistik seperti total temuan, jumlah match per algoritma, waktu eksekusi, dan perbandingan keyword ditampilkan melalui popup.

Secara keseluruhan, Judol Detector telah memenuhi spesifikasi utama tugas besar, yaitu membangun extension berbasis TypeScript yang menerapkan KMP, Boyer-Moore, RegEx, dan Weighted Levenshtein Distance untuk mendeteksi konten judi online pada halaman web.

5.2 Saran

Pengembangan selanjutnya dapat dilakukan dengan memperluas cakupan deteksi pada halaman web yang lebih kompleks, seperti teks di dalam iframe, shadow DOM, atau konten yang dimuat secara dinamis setelah halaman dibuka.

Threshold pada fuzzy matching juga dapat diuji dan disesuaikan lebih lanjut agar hasil deteksi tetap akurat. Threshold yang terlalu rendah dapat menyebabkan false positive, sedangkan threshold yang terlalu tinggi dapat menyebabkan variasi teks tertentu tidak terdeteksi.

Selain itu, performa algoritma dapat dianalisis lebih dalam pada halaman dengan jumlah teks yang besar dan daftar keyword yang lebih banyak. Optimasi dapat dilakukan agar proses deteksi tetap cepat tanpa mengurangi akurasi.

LAMPIRAN

Link Github:

https://github.com/jonveral/Tubes3_sayakanlawan.git

Link YouTube:

<https://youtu.be/c82fAy4rMUc>

Tabel Pembagian Tugas

NIM	Nama	Tugas
13524092	Timothy Bernard Soeharto	Setup awal dan arsitektur extension, meliputi konfigurasi TypeScript/build, manifest.json (entry point untuk web browser ke program), keyword loader, kontrak data, pipeline deteksi, content script, dan struktur awal popup
13524120	Jonathan Alveraldo Bangun	Implementasi algoritma Knuth-Morris-Pratt, implementasi algoritma Boyer-Moore, Implementasi comparison counting, failure function, last occurrence table, shifting process, implementasi algoritma Aho-Corasick (bonus), implementasi algoritma Rabin-Karp (bonus), Pengujian (Testing) Algoritma
13524126	Ramadhian Nabil Firdaus Gumay	Implementasi regexMatcher, implementasi algoritma weightedLevenshtein, implementasi highlighting, implementasi popup, implementasi custom tooltip yang meliputi keyword, algoritma, jumlah, dan runtime, implementasi visualisasi statistik popup, implementasi bonus toggle blur dan blur teks yang terdeteksi

No	Poin	Ya	Tidak
1	Extension berhasil di-build dan di-load tanpa kesalahan pada chromium browser dan dikembangkan dengan TypeScript	✓	
2	KMP dan Boyer-Moore diimplementasikan from scratch	✓	
3	Regex handle format <kata><angka> dan berbagai edge case	✓	
4	Pencarian KMP & BM membaca keyword.txt secara iteratif dan tidak menggunakan built-in search function atau library eksternal	✓	
5	Exact matching dan fuzzy matching berjalan benar	✓	
6	Elemen DOM terdeteksi diberi highlight dan terhapus saat rescanning	✓	
7	Tooltip muncul saat hover dengan informasi keyword, algoritma, kemunculan, dan waktu eksekusi	✓	
8	Popup menampilkan statistik realtime (total keyword, perbandingan, waktu eksekusi, jumlah match)	✓	
9	[Bonus] Membuat Video	✓	
10	[Bonus] Implementasi Algoritma Aho-Corasick dan Rabin Karp	✓	
11	[Bonus] Implementasi Censorship / Blur Teks	✓	
12	[Bonus] Implementasi Optical Character Recognition pada Gambar		✓

DAFTAR PUSTAKA

1. Chrome Extension Documentation

<https://developer.chrome.com/docs/extensions/>

2. Tesseract.js

<https://tesseract.projectnaptha.com/>

3. Aho-Corasick

<https://www.geeksforgeeks.org/dsa/aho-corasick-algorithm-pattern-searching/>

4. Rabin Karp

<https://www.geeksforgeeks.org/dsa/rabin-karp-algorithm-for-pattern-searching/>

5. Create a Chrome extension from scratch step-by-step (2024)

<https://youtu.be/2dQJYDAAU4I?si=llJ5eyyL-cHCP996>